

Appendix of ProSpec: Value-Guided Prospective Action Filtering for Data-Efficient Model-Free Reinforcement Learning

Liangliang Liu^{ID}, Yi Guan, Rujia Shen^{ID}, Chaoran Kong, Guowei Zheng^{ID},
Jingchi Jiang^{ID} and Yi Lin^{ID}



APPENDIX A SYMBOLS DEFINITION

For clarity, we have listed all the symbols used throughout the paper and their meanings in Table 1.

TABLE 1
Definitions of the symbols used in this document.

Symbol	Definition
$\mathcal{S}$	the set of states
$\mathcal{A}$	action space
$\mathcal{T}(\cdot, \cdot)$	dynamic function
$\mathcal{R}(\cdot, \cdot)$	reward function
γ	discount factor
$G_t(\cdot)$	cumulative discounted return
$Q_\pi(\cdot)$	action-value function
π^*	optimal policy
a	action
s	state
$\tilde{z}$	latent state
$\hat{z}$	dynamic model forward predicts latent state
$\tilde{z}$	dynamic model backward predicts latent state
$sc(\cdot)$	scaling function
$t(\cdot)$	translation function
CQ	cumulative discounted value
$\mathcal{O}$	matrix family
k	the count of prospective
t	prediction horizon (time)
N	the range of time (MPC)
U	total time in forward prediction loss
W	weight
I	identity matrix
B	bias
M	number of sampled actions in cyclical consistency loss
$h(\cdot, \cdot)$	FDM forward calculation function
$h^{-1}(\cdot, \cdot)$	FDM backward calculation function
$q(\cdot)$	prediction head
$g(\cdot)$	online projection heads
$g_m(\cdot)$	target projection heads
$d(\cdot, \cdot)$	distance metrics

APPENDIX B ALGORITHM

B.1 Soft Actor Critic

Soft Actor-Critic (SAC) [1] is a widely used gradient-based algorithm for continuous control that incorporates policy entropy as an additional reward to encourage exploration. Let θ denote the parameters of the stochastic policy π and ϕ denote the parameters of $Q(s_t, a_t)$. SAC tries to learn a probabilistic policy π_θ , two Q functions Q_{ϕ_1} and Q_{ϕ_2} , and a temperature parameter α to adjust the balance between exploration and exploitation.

Critic. To mitigate the risk of overestimation, SAC adopts a twin Q-network structure and chooses the minimum estimate from the two Q-functions. SAC constructs the target loss function for Critic updates using transitions sampled from the experience replay buffer $(s_t, a_t, s_{t+1}, r_t, done)$:

$$\mathcal{L}_\phi^{\text{critic}} = \sum_{i=1,2} (Q_{\phi_i}(s_t, a_t) - y(r_t, s_{t+1}, done))^2 \quad (1)$$

where r_t denotes the reward, s_{t+1} represents the next state, and $done$ is the termination flag, with a value of 1 indicating that s_{t+1} is a terminal state and 0 otherwise. The target is the clipped gradient, defined as:

$$y(r_t, s_{t+1}, done) = r_t + \gamma(1 - done)(\min_{i=1,2} Q_{\phi_{target,i}}(s_{t+1}, a_{t+1} - \alpha \log_{\phi_\theta}(a_{t+1}|s_{t+1}))) \quad (2)$$

where γ is the discount factor and $Q_{\phi_{target,i}}$ is the target Q-network updated using the exponential moving average (EMA) of Q_{ϕ_i} .

Actor. The objective of the actor π_θ is to select actions that maximize the action value and policy entropy. Therefore, the update loss function for the actor can be defined as:

$$\mathcal{L}_\theta^{\text{actor}} = - \left(\min_{i=1,2} Q_{\phi_i}(s_t, a_\theta(s_t)) - \alpha \log \pi_\theta(a_\theta(s_t) | s_t) \right) \quad (3)$$

where $a_\theta(s_t)$ is sampled from a stochastic policy $\pi_\theta(s_t)$. The overall objective. The overall training objective of SAC is as follows:

$$\mathcal{J}_\theta^{\text{SAC}} = \mathcal{L}_\phi^{\text{critic}} + \mathcal{L}_\theta^{\text{actor}} + \mathcal{L}_\alpha \quad (4)$$

where, $\mathcal{L}_\phi^{\text{critic}}$, $\mathcal{L}_\theta^{\text{actor}}$ and $\mathcal{L}_\alpha$ represent the critic loss, actor loss and temperature loss, respectively.

B.2 DQN

The Deep Q-Network (DQN) [2] encodes the state through a neural network and outputs the corresponding Q-values for the actions, thereby enabling Q-learning to scale to large state spaces. Additionally, DQN incorporates techniques such as experience replay and target network separation to stabilize the training process. The overall objective of DQN is represented by the following loss function:

$$\mathcal{J}_\theta^{\text{DQN}} = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a') - Q_\theta(s, a) \right)^2 \right] \quad (5)$$

Where, Q_θ denotes the Q network controlled by parameters θ , and $Q_{\bar{\theta}}$ denotes the target Q network. Meanwhile, $\mathcal{D}$ denotes the replay buffer storing the experience tuples.

APPENDIX C

MORE IMPLEMENTATION DETAIL

C.1 Network architecture for DMControl

Considering that the original SPR was designed for discrete control tasks, we have developed an SPR-like scheme, called $\text{SPR}^\dagger$, for continuous control tasks, following previous research [3], [4]. Specifically, we use the encoder and policy networks of CURL [5] as our base networks. Building on SPR [6], we remove the contrast loss from CURL and introduce a BYOL [7] head to construct our SPR-like baseline scheme. For the dynamics model (DM) within $\text{SPR}^\dagger$, we adopt a network structure similar to that used in DBC [8]. The DM consists of two fully connected layers, with a linear normalization (LN) layer and a ReLU activation following the first fully connected layer. The encoder consists of four convolutional layers (each followed by a ReLU), followed by a fully connected layer, an LN layer, and a hyperbolic tangent (tanh) activation. Similar to the design in SPR, we use a projection head $g(\cdot)$, a prediction head $q(\cdot)$ for the online encoder, an impulse encoder $f_m(\cdot)$, and an impulse projection head $g_m(\cdot)$. Both the projection head and the prediction head consist of two fully connected layers (preceded by a ReLU layer), each containing 512 hidden units.

C.2 FDM Prediction Objective in Atari

In the MDP framework, a one-step transition (s_t, a_t, s_{t+1}) describes how the agent moves from the current state $s_t \in \mathcal{S}$ to the next state $s_{t+1} \in \mathcal{S}$ by taking action $a_t \in \mathcal{A}$. For Atari tasks, the high-dimensional visual observations contain rich appearance variations and task-irrelevant details, which makes stable latent transition prediction important for generating reliable prospective rollouts. In ProSpec, the flow-based reversible dynamics model (FDM) $h(\cdot, \cdot)$ is used to approximate the environment transition in the latent space. Given the current observation s_0 , the encoder $f(\cdot)$ first maps it into a latent representation $z_0 = f(s_0)$. Then, conditioned on the action $\hat{a}_{t-1}$, FDM recursively predicts the future latent state:

$$\hat{z}_t = \begin{cases} z_0 = f(s_0), & t = 0, \\ h(\hat{z}_{t-1}, \hat{a}_{t-1}), & t > 0, \hat{a}_{t-1} \sim \pi. \end{cases} \quad (6)$$

The objective of FDM is to align the predicted latent state $\hat{z}_{t+u}$ with the target latent state $z_{t+u} = f(s_{t+u})$ encoded from the corresponding future observation. Following PlayVirtual [3], we apply a projection head in Atari and compute the prediction loss in the projected latent space. This design avoids forcing the model to directly match all raw latent features and provides a more stable training signal under visually complex Atari observations. The prediction objective is formulated as:

$$\mathcal{L}_{\text{pred}} = - \sum_{u=1}^U d(g(\hat{z}_{t+u}), g(z_{t+u})), \quad (7)$$

where $g(\cdot)$ denotes the projection head, $d(\cdot, \cdot)$ denotes the similarity measure in the projected latent space, and U is the prediction horizon. The projection head is used only for optimizing the FDM prediction objective and does not affect model-free policy execution.

Through this prediction objective, FDM learns to generate latent prospective rollouts in Atari. These rollouts are not treated as real trajectories for policy or value updates. Instead, they provide rollout-level evidence for VPAF, which ranks candidate first-step actions using cumulative Q-value scores before real-environment execution.

C.3 The Flow-based Dynamics Model (FDM)

The FDM predicts the future and retroactively infers previous latent states, crucial in ProSpec. Unlike PlayVirtual [3], which employs separate models for these tasks, our approach employs a single model for both, avoiding training instability. We utilize Real-valued Non-Volume Preserving (RealNVP) [9], a flow-based neural network that performs bijective coupling transformations, which can be seen in the Fig. 2a. RealNVP operates via bijective coupling layers, with each layer handling parts of the input $a_t \in \mathbb{R}^{1:d}$ and $z_t \in \mathbb{R}^{d+1:D}$ separately. It applies transformations using affine functions for scaling $\exp(sc_i(\cdot))$ and translation $t_i(\cdot)$, along with element-wise multiplication ' $\odot$ ' and addition '+' operations. This enables the computation of the forward process as:

$$\begin{aligned} \hat{z}_{t+1}^{1:d} &= \hat{a}_t \odot \exp(sc_2(\hat{z}_t)) + t_2(\hat{z}_t) \\ \hat{z}_{t+1}^{d+1:D} &= \hat{z}_t \odot \exp(sc_1(\hat{z}_{t+1}^{1:d})) + t_1(\hat{z}_{t+1}^{1:d}) \end{aligned} \quad (8)$$

and the backward processes can be calculated as:

$$\begin{aligned} \hat{z}_t &= (\hat{z}_{t+1}^{d+1:D} - t_1(\hat{z}_{t+1}^{1:d})) \odot \exp(-sc_1(\hat{z}_{t+1}^{1:d})) \\ \hat{a}_t &= (\hat{z}_{t+1}^{1:d} - t_2(\hat{z}_t)) \odot \exp(-sc_2(\hat{z}_t)) \end{aligned} \quad (9)$$

As shown in Fig. 2a, there is a dimensional mismatch between the quantities $\hat{z}_t \in \mathbb{R}^D$ and $\hat{z}'_t \in \mathbb{R}^{d+1:D}$, $\hat{z}'_{t+1} \in \mathbb{R}^{d+1:D}$ and $\hat{z}_{t+1} \in \mathbb{R}^D$, computed by RealNVP. Typically, a linear transformation of a feed-forward neural network can be expressed as:

$$\begin{aligned} \hat{z}'_t &= W \times \hat{z}_t + B \\ \hat{z}'_{t+1} &= W \times \hat{z}_{t+1} + B \end{aligned} \quad (10)$$

However, due to the typical non-invertibility of W , backward inference (i.e., directly computing $\hat{z}_{t+1} = W^{-1} \times (\hat{z}'_{t+1} - B)$) presents challenges. To ensure reversibility, we employ Orthogonal Weight Normalization (OWN) [10],

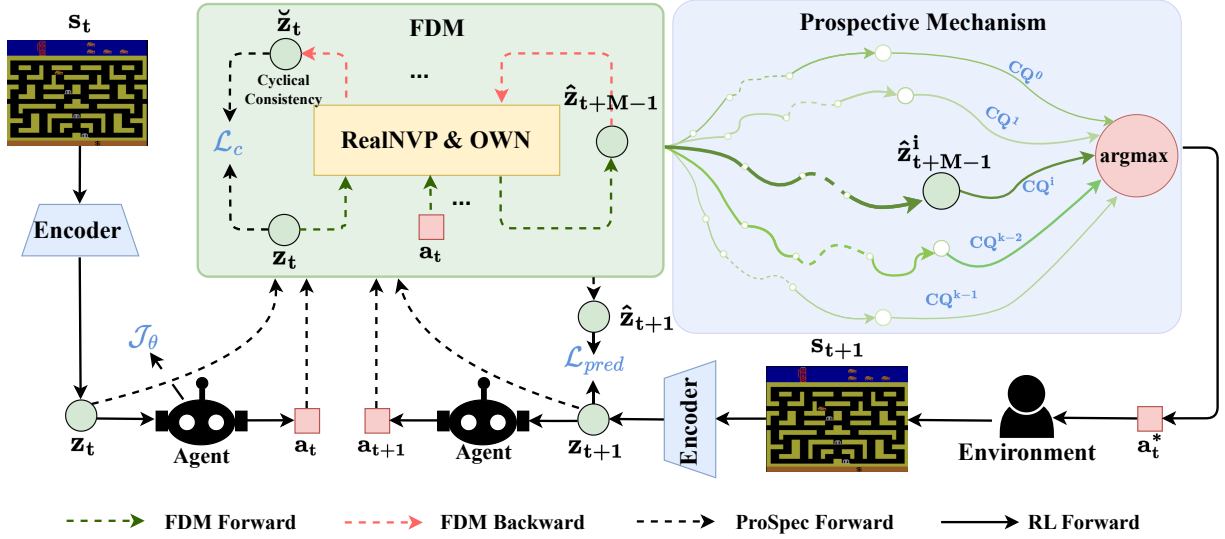


Fig. 1. Training Procedure of ProSpec. Here, $\mathcal{J}_\theta$ represents the RL loss; $\mathcal{L}_{pred}$ denotes the prediction loss of the FDM; $\mathcal{L}_c$ stands for the cyclical consistency loss.

which enables bidirectional inference by enforcing orthogonality in the weights. As a result, the backward inference of $\tilde{z}_t$ can be re-expressed as:

$$\begin{aligned} \hat{z}_t &= W^T \times (\hat{z}'_t - B) \\ \tilde{z}_{t+1} &= W^T \times (\tilde{z}'_{t+1} - B), WW^T \in \mathcal{O}^{D \times D} \end{aligned} \quad (11)$$

where the matrix family $\mathcal{O}^{D \times D} = \{W \in \mathbb{R}^{(d+1:D) \times D} \mid WW^T = W^{-1}W = I\}$ (see [10] for more details). For clarity, we have summarized the reasoning process of OWN in Fig. 2b. By using the combination of RealNVP and OWN, we can effectively achieve bidirectional transformation between $\tilde{z}_t$ and $\tilde{z}_{t+1}$.

C.4 Hyperparameter Settings

We list the hyperparameters used for the DMControl and Atari benchmarks in Table 10 and Table 11, mainly following the CURL settings [5]. The entire process is implemented using PyTorch, and network training is performed on an NVIDIA RTX 3090 GPU.

APPENDIX D

MORE EXPERIMENTAL RESULTS AND ANALYSIS

D.1 Applicability Analysis of ProSpec

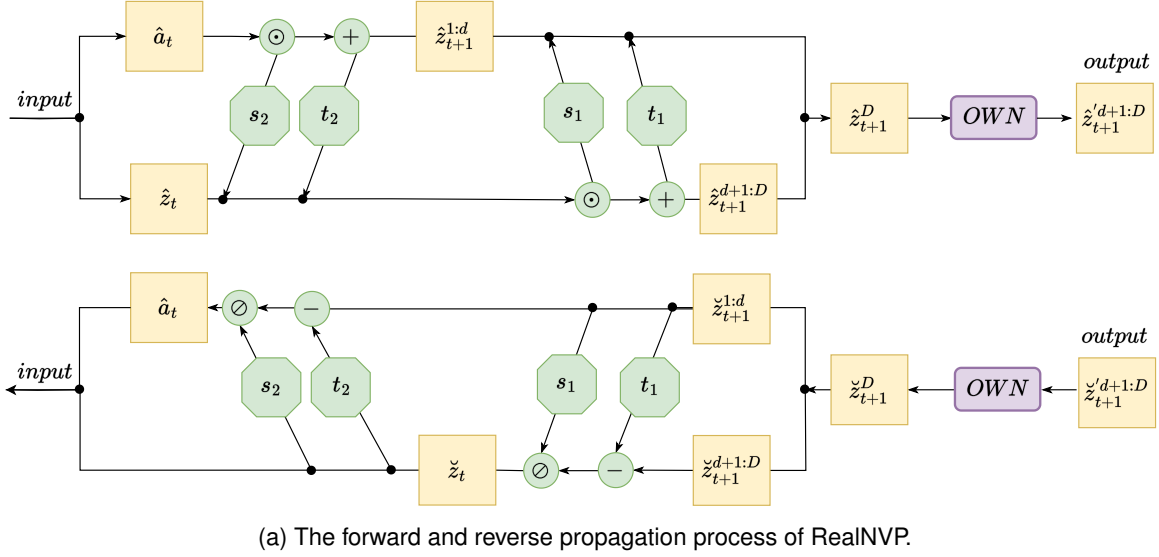
To further analyze the applicability of ProSpec, we conduct a qualitative comparison with the strong PlayVirtual baseline on several Atari games. Both methods are trained for 100,000 iterations under the same random seed and hyperparameter configuration, and the corresponding checkpoints are used for downstream behavioral analysis. We initialize each environment to a fixed predefined state in representative games, including *Bank Heist*, *Chopper Command*, and *Amidar*. For each checkpoint, we record the action selected by the learned policy or value network and the resulting next state, as shown in Fig. 3. We focus on Atari because its visual states and action consequences are more interpretable than those in DMControl.

It should be emphasized that this analysis evaluates the learned policy after training. During evaluation, the FDM-based prospective branch and VPAF are disabled, and actions are directly produced by the learned model-free policy or value network. Therefore, the observed behavioral differences reflect the influence of training-time prospective value-aware interaction on the learned policy, rather than online model-based planning during inference.

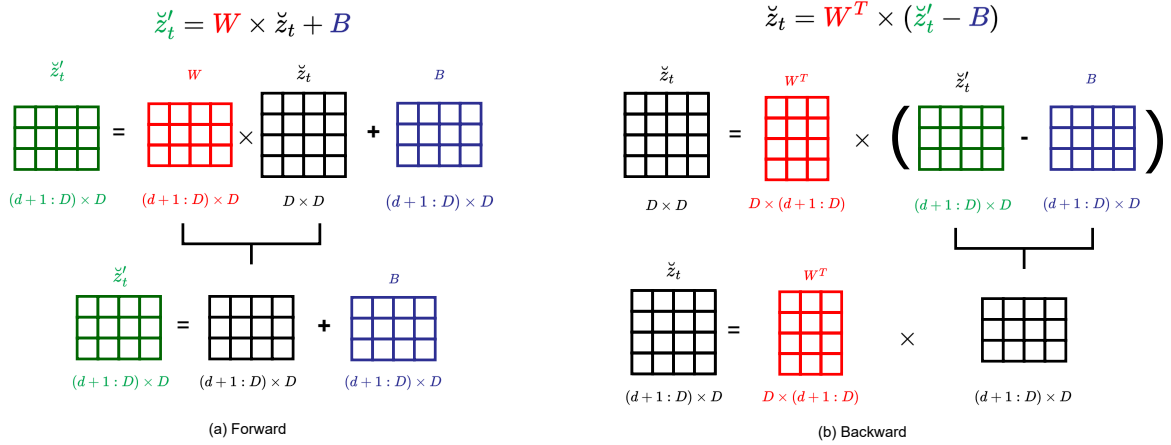
Delayed-consequence decisions. In *Bank Heist*, PlayVirtual tends to choose an action that moves directly toward the immediate reward, which may expose the agent to approaching guards and lead to an unfavorable next state. In contrast, the policy learned by ProSpec selects an action that avoids the immediate threat and preserves more favorable future options. This behavior is consistent with the design of VPAF, which reduces low-value action execution during training by ranking candidate actions according to FDM-generated latent prospective rollouts.

Avoiding hard-to-recover transitions. In *Chopper Command*, an inappropriate action may quickly lead to collision or failure. As shown in Fig. 3b, when the helicopter approaches ground targets, PlayVirtual selects an action that results in a less favorable next state, whereas ProSpec selects an action that better preserves control and survivability. This difference suggests that training-time prospective value-aware interaction can help the learned policy avoid actions with poor delayed consequences.

Sparse-reward goal-driven behavior. In *Amidar*, rewards are sparse and ineffective exploration can slow down policy improvement. As shown in Fig. 3c, PlayVirtual selects an action that moves the agent toward a more crowded region, increasing the risk of an unfavorable transition. By contrast, ProSpec selects an action that avoids immediate danger and maintains a more favorable position for future reward collection. This observation is consistent with the quantitative results showing that ProSpec improves data efficiency in tasks where immediate rewards provide limited guidance.



$$a_t \in \mathbb{R}^{(1:d) \times D}, \hat{z}_{t+1} \in \mathbb{R}^{D \times D}, W \in \mathbb{R}^{D \times (d+1:D)}, B \in \mathbb{R}^{D \times (d+1:D)}$$



(b) The reasoning process of OWN.

Fig. 2. Flow-based Reversible Dynamics Model of ProSpec.

In dense-reward and highly reactive environments such as *Freeway*, the benefit of ProSpec is less pronounced because standard trial-and-error learning already provides sufficiently informative feedback for policy improvement. Overall, these qualitative results suggest that ProSpec is particularly useful in tasks where delayed consequences, sparse rewards, or hard-to-recover transitions make purely reactive data collection less efficient. They further support the role of VPAF in improving the quality of training-time interactions while preserving model-free inference.

D.2 More Ablation Studies

We conduct additional ablation experiments to further examine the effectiveness of the key components of ProSpec. Considering the variance across environments and the computational cost of large-scale RL evaluation, we perform experiments on DMControl and Atari with 5 random seeds.

Effectiveness of VPAF. We first evaluate the effectiveness of the proposed Value-Guided Prospective Action Filter (VPAF) by comparing ProSpec with ProSpec-NP. ProSpec-NP denotes the variant in which VPAF is disabled, and the agent collects real-environment interactions directly using the underlying model-free backbone. As shown in Table 2 and Table 3, ProSpec achieves consistent improvements over ProSpec-NP across most environments. These results indicate that converting trial-and-error data collection into prospective value-aware interaction provides additional benefits beyond the underlying model-free backbone.

Specifically, ProSpec obtains multiple candidate first-step actions from the model-free backbone, uses FDM to generate their latent prospective rollouts, and ranks these rollouts using cumulative Q-value scores. Only the first action of the highest-scored rollout is executed for real-environment interaction and replay collection. Compared

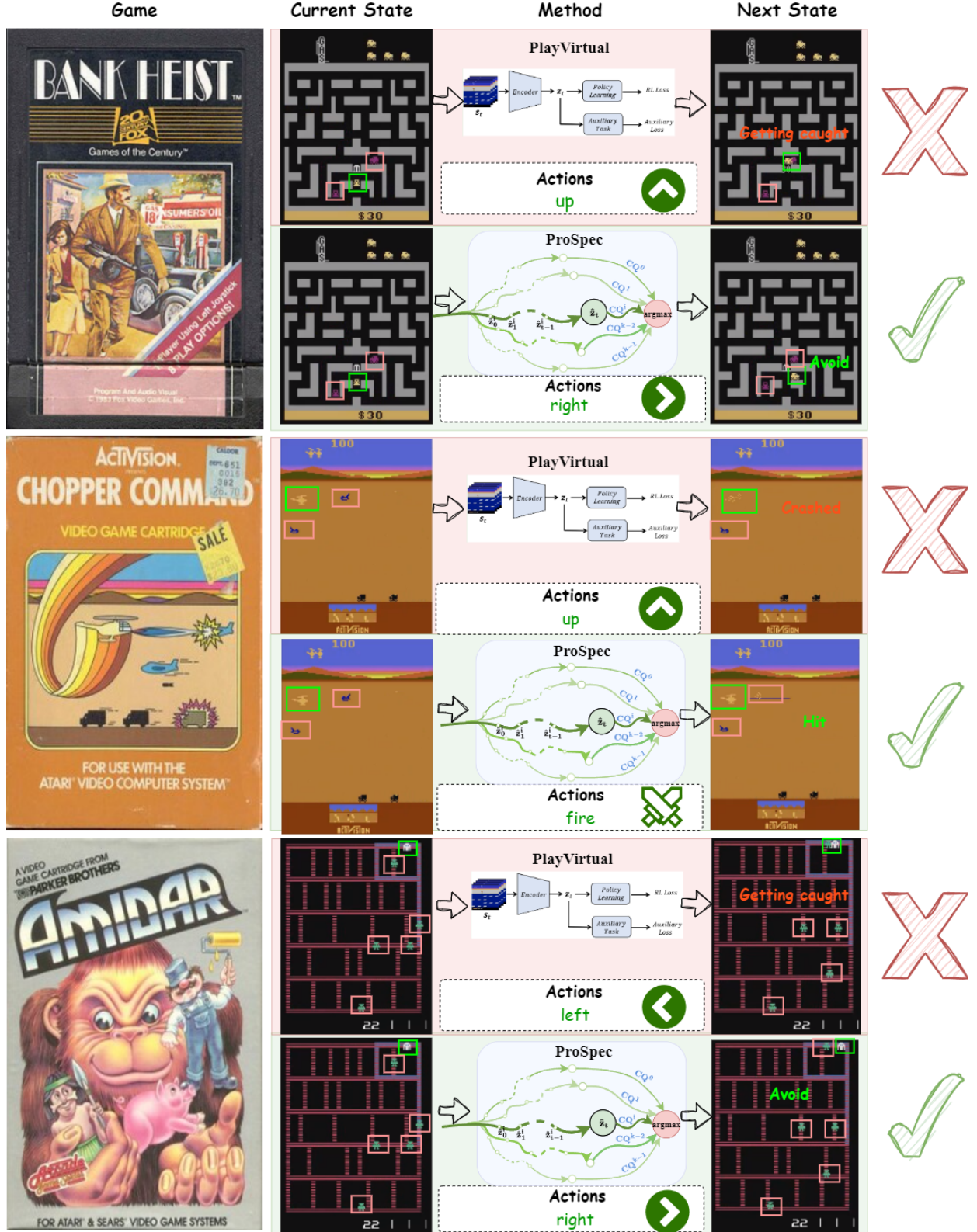


Fig. 3. ProSpec vs. PlayVirtual Decision-Making Examples in Atari Games (Bank Heist, Chopper Command, Amidar).

with ProSpec-NP, which relies on standard model-free data collection, ProSpec can reduce low-value action execution and improve the quality of collected transitions. Therefore,

the performance gains confirm that VPAF improves data efficiency by guiding training-time interaction with prospective value-aware action ranking while preserving model-

free inference during evaluation.

TABLE 2
Effectiveness of VPAF in DMControl benchmark.

	ProSpec-NP	ProSpec
Finger, spin	843±48	875±104
Cartpole, swingup	816±41	833±34
Reacher, easy	688±28	782±137
Cheetah, run	459±22	477±24
Walker, walk	455±56	507±60
Ball in cup, catch	884±68	927±17
Median Score	752	807.5

TABLE 3
Effectiveness of VPAF in Atari benchmark.

Game	ProSpec-NP	ProSpec
Alien	794.9	862.8
Amidar	151.2	170.6
Assault	648.6	703.6
Asterix	958.3	957.4
Bank Heist	317.7	481.1
Battle Zone	12,334	15,934.4
Boxing	43.7	41.5
Breakout	18.8	20.4
Chopper Command	709	1,351
Crazy Climber	28,505.8	38,803
Demon Attack	898.9	963.5
Freeway	26.9	25.1
Frostbite	537.3	1864.6
Gopher	747.9	866
Hero	6,698.4	8,140.5
Jamesbond	351.5	384
Kangaroo	3,281	2,494
Krull	3,842.7	4,542.2
Kung Fu Master	17,756.2	20,711
Ms Pacman	1,202.7	1,139.7
Pong	-4.6	-1.1
Private Eye	100	100
Qbert	2,920.1	3,950
Road Runner	8,172.3	13744
Seaquest	607.3	800
Up N Down	8,977.3	16,640
IQM HNS (%)	43.5	47.2

TABLE 4
Impact of Rollout Horizon H .

Benchmark	H=1	H=3	H=6	H=9	H=12	H=15
DMControl	754.4	726.6	809.0	731.4	771.3	774.9
Atari	42.4	44.1	38.1	36.3	36.4	34.2

Impact of Candidate Count K . We investigate the influence of the candidate count K , which denotes the number of candidate first-step actions evaluated by VPAF during training. For each candidate action, FDM generates a corresponding latent prospective rollout, and VPAF computes its cumulative Q-value score for ranking. Experiments are conducted with $K \in \{1, 3, 6, 9\}$, and the median results are reported across different rollout horizons $H \in \{1, 3, 6, 9, 12, 15\}$. As

TABLE 5
Impact of Candidate Count K in Atari-100k. Note that since Gopher has only 8 operable actions, the result for $K = 9$ is recorded as the result for $K = 8$.

Game	K=1	K=3	K=6	K=9
Alien	868.4	764.1	792.9	849.3
Amidar	149.5	115.8	132.5	208.7
Assault	629.8	610.8	650.1	653.0
Asterix	960.5	897.8	853.8	890.4
Bank Heist	273.8	225.0	258.0	585.1
Battle Zone	13,350.0	11,810.9	13,818.3	16,213.3
Boxing	35.6	28.1	35.4	41.1
Breakout	20.3	14.9	15.6	14.5
Chopper Command	597.0	860.9	1002.5	1064.3
Crazy Climber	25,691.0	22,536.2	19,337.3	15,873.0
Demon Attack	922.1	779.7	692.4	833.1
Freeway	29.4	26.9	27.7	23.6
Frostbite	267.8	1,805.0	1,990.9	1,632.7
Gopher	646.4	643.7	698.8	684.8
Hero	6,094.8	7,499.4	9,843.3	8,960.0
Jamesbond	373.5	307.4	336.9	348.6
Kangaroo	4,364.0	4,131.3	1,634.0	4,775.0
Krull	3,830.4	3,579.0	3,645.4	4,320.2
Kung Fu Master	14,497.0	16,651.1	16,439.8	12,952.8
Ms Pacman	1,154.5	964.6	980.6	1,041.3
Pong	-8.4	3.9	-12.7	-13.0
Private Eye	100.0	100.0	105.5	100.0
Qbert	2,155.3	1,975.6	1,714.1	3,519.9
Road Runner	6,343.0	10,000.8	10,941.9	7,727.5
Seaquest	640.8	582.5	664.8	693.6
Up N Down	5,118.9	6,345.5	9,454.4	10,418.4
IQM HNS (%)	37.3	39.7	37.3	41.4

summarized in Table 9, increasing K generally improves performance compared with $K = 1$, indicating that comparing multiple candidate actions before real-environment execution benefits prospective value-aware interaction.

In DMControl, the best performance is achieved at $K = 3$, with a 6.9% improvement over $K = 1$ and a 1.9% improvement over $K = 9$. This suggests that a moderate number of candidate actions is sufficient for continuous-control tasks, where further increasing K may introduce additional computation without providing substantially more discriminative candidates. In Atari, the highest performance is obtained at $K = 9$, showing an 11.0% improvement over both $K = 1$ and $K = 6$. This may be because Atari has a finite discrete action space, where evaluating more top-ranked candidate actions can better identify actions with higher estimated long-term value.

Overall, the performance gain does not scale linearly with K . When K becomes large, different candidate actions may yield similar cumulative Q-value scores, leading to marginal additional benefits while increasing training-time computation. Therefore, K controls a trade-off between candidate action diversity and the computational cost of VPAF.

Impact of Rollout Horizon H . We further investigate the impact of the rollout horizon H , which determines the number of future latent steps generated by FDM for evaluating each candidate first-step action. Similar to the previous experiment, we report the median performance across different candidate counts K , as shown in Table 4. It should be noted that the FDM-generated rollout is used only for cumulative Q-value scoring, and VPAF retains only the first action of the highest-scored rollout for real-environment interaction and replay collection.

TABLE 6
Impact of Rollout Horizon H on ProSpec in Atari-100k.

Game	H=1	H=3	H=6	H=9	H=12	H=15
Alien	931.2	831.4	791.9	753.5	727.1	681.0
Amidar	168.5	152.2	160.1	132.1	125.6	155.1
Assault	686.3	663.6	660.9	678.3	642.2	460.4
Asterix	911.5	856.6	853.5	819.1	816.2	881.3
Bank Heist	272.8	372.4	493.4	180.3	181.5	303.0
Battle Zone	10,808.0	15,218.3	13,303.3	13,377.5	12,910.0	9,203.3
Boxing	35.1	32.7	43.7	35.4	39.2	19.9
Breakout	17.4	22.2	13.5	13.5	12.9	10.0
Chopper Command	729.0	1161.7	912.0	795.4	932.3	863.3
Crazy Climber	23,838.6	20,409.4	20,094.8	15,210.6	19,538.8	14,857.7
Demon Attack	793.3	902.0	852.8	737.3	638.9	463.2
Freeway	28.8	27.4	25.2	27.0	27.1	23.1
Frostbite	1,446.1	1,937.2	1,498.8	1,354.1	1,532.2	1,002.1
Gopher	710.1	836.5	721.0	669.8	517.5	422.3
Hero	7,962.0	5,874.2	8,426.9	8,408.5	9,537.9	6,335.0
Jamesbond	363.4	370.7	338.7	309.2	315.8	280.3
Kangaroo	2,337.8	2,461.3	3,058.0	3,993.3	2,896.7	4,454.0
Krull	4,136.2	4,296.7	3,450.2	3,690.2	3,322.8	3,966.3
Kung Fu Master	16,073.1	18,646.5	13,194.3	17,548.7	20,366.7	11,413.0
Ms Pacman	1,134.1	1,192.6	921.5	869.1	804.8	932.4
Pong	-5.4	-5.8	-10.3	-8.7	-8.2	-3.7
Private Eye	100.0	100.0	100.0	100.0	111.0	100.0
Qbert	3,307.4	3,019.5	1,833.8	2,189.2	760.0	1,070.6
Road Runner	9,833.1	7,431.2	9,314.3	11,405.6	11,167.7	7,705.7
Seaquest	605.3	711.3	604.5	614.5	682.1	677.3
Up N Down	17,462.2	7,222.8	2,907.5	4,238.3	9,431.6	4,767.2
IQM HNS (%)	42.4	44.1	38.1	36.3	36.4	34.2

TABLE 7
Impact of Candidate Count K in DMControl-100k.

Game	K=1	K=3	K=6	K=9
Finger, spin	880.0	867.9	871.2	811.7
Cartpole, swingup	816.0	831.5	820.0	800.0
Reacher, easy	675.8	762.6	770.3	763.6
Cheetah, run	349.8	388.2	384.5	419.7
Walker, walk	424.6	561.4	517.6	516.5
Ball in cup, catch	845.1	905.5	905.8	934.4
Median Score	745.9	797.0	795.1	781.8

TABLE 8
Impact of Rollout Horizon H on ProSpec in DMControl-100k.

Game	H=1	H=3	H=6	H=9	H=12	H=15
Finger, spin	877.6	813.0	820.6	785.3	772.3	810.4
Cartpole, swingup	813.0	793.0	818.0	817.0	829.0	836.0
Reacher, easy	695.8	670.4	785.2	675.3	766.7	699.5
Cheetah, run	361.7	375.3	466.5	415.3	368.6	382.9
Walker, walk	522.8	573.7	581.4	555.4	522.4	539.0
Ball in cup, catch	868.7	908.1	927.2	926.5	921.8	918.0
Median Score	754.4	726.6	809.0	731.4	771.3	774.9

TABLE 9
Impact of Candidate Count K .

Benchmark	K=1	K=3	K=6	K=9
DMControl	745.9	797.0	795.1	781.8
Atari	37.3	39.7	37.3	41.4

important for balancing prospective information and rollout reliability. In DMControl, ProSpec achieves the best performance at $H = 6$, with a 7.2% improvement over $H = 1$ and a 4.4% improvement over $H = 15$. In Atari, the best performance is obtained at $H = 3$, with a 4.0% improvement over $H = 1$ and a 28.9% improvement over $H = 15$. These results indicate that a longer rollout horizon does not necessarily lead to better performance. A short horizon may provide insufficient future information for distinguishing candidate first-step actions, whereas an overly long horizon may introduce larger FDM prediction errors and reduce the reliability of cumulative Q-value ranking. Therefore, an intermediate horizon provides a better trade-off between prospective information and rollout reliability.

Overall. Tables 5–8 present the detailed effects of the candidate count K and rollout horizon H across different environments, and the summarized results are reported in Tables 9 and 4. Based on these results, we set $K = 3$

The results show that an appropriate rollout horizon is

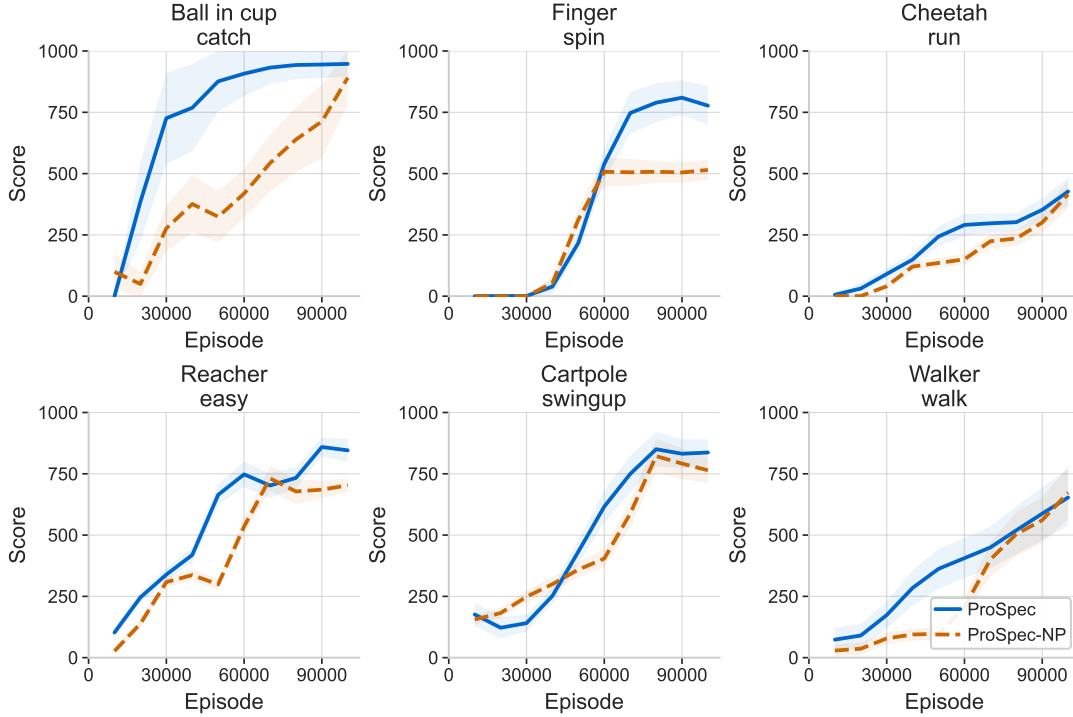


Fig. 4. The impact of ProSpec on data efficiency (DMControl: Ball in cup to Walker).

and $H = 6$ for DMControl, and $K = 9$ and $H = 3$ for Atari. These choices reflect the different characteristics of continuous and discrete control tasks. DMControl benefits from a moderate number of candidate actions and a relatively longer rollout horizon, whereas Atari benefits from evaluating more discrete candidate actions but requires a shorter horizon to reduce error accumulation in visually complex latent rollouts. Overall, the ablation results confirm that both K and H are important for balancing candidate action diversity, rollout reliability, and the computational cost of VPAF.

D.3 Impact of ProSpec on Data Efficiency

We investigate the impact of ProSpec on convergence speed and data efficiency using the DMControl and Atari benchmarks. Specifically, we compare ProSpec with ProSpec-NP, where VPAF is disabled and the agent collects real-environment interactions directly using the underlying model-free backbone. For each environment, we record the evaluation rewards or game scores throughout training to analyze the learning dynamics.

As shown in Fig. 4–7, ProSpec achieves substantially higher scores than ProSpec-NP during the early training stage, demonstrating improved data efficiency. In terms of convergence speed, ProSpec reaches competitive or near-optimal performance with fewer environment interactions. For example, in the *Amidar* environment, ProSpec surpasses ProSpec-NP within fewer than 10,000 training steps, whereas ProSpec-NP requires considerably more interactions to reach a comparable score.

The faster convergence of ProSpec can be attributed to VPAF. Instead of relying solely on trial-and-error data

collection, ProSpec obtains multiple candidate first-step actions from the model-free backbone, uses FDM to generate their latent prospective rollouts, and ranks these rollouts using cumulative Q-value scores. Only the first action of the highest-scored rollout is executed for real-environment interaction and replay collection. This process converts trial-and-error data collection into prospective value-aware interaction, enabling ProSpec to reduce low-value action execution and improve the quality of collected transitions.

In contrast, ProSpec-NP removes VPAF and relies only on the original model-free data collection strategy. As a result, it is more likely to collect low-value or less informative transitions, leading to slower learning and larger score fluctuations, especially in complex or sparse-reward environments. In terms of final performance, ProSpec also outperforms ProSpec-NP in most environments, suggesting that VPAF not only accelerates convergence but also improves the quality of the learned policy.

Overall, these results demonstrate that ProSpec improves both data efficiency and final performance by converting training-time interaction into prospective value-aware interaction. During inference, VPAF and the FDM-based prospective branch are disabled, and actions are directly produced by the model-free backbone, thereby preserving model-free inference.

D.4 Training Cost and Inference Efficiency

ProSpec augments model-free RL backbones with VPAF only during training. The additional computational cost mainly comes from generating K FDM-based latent prospective rollouts with horizon H and evaluating these rollouts with the target critic for cumulative Q-value scoring. For each training state, the additional theoretical cost

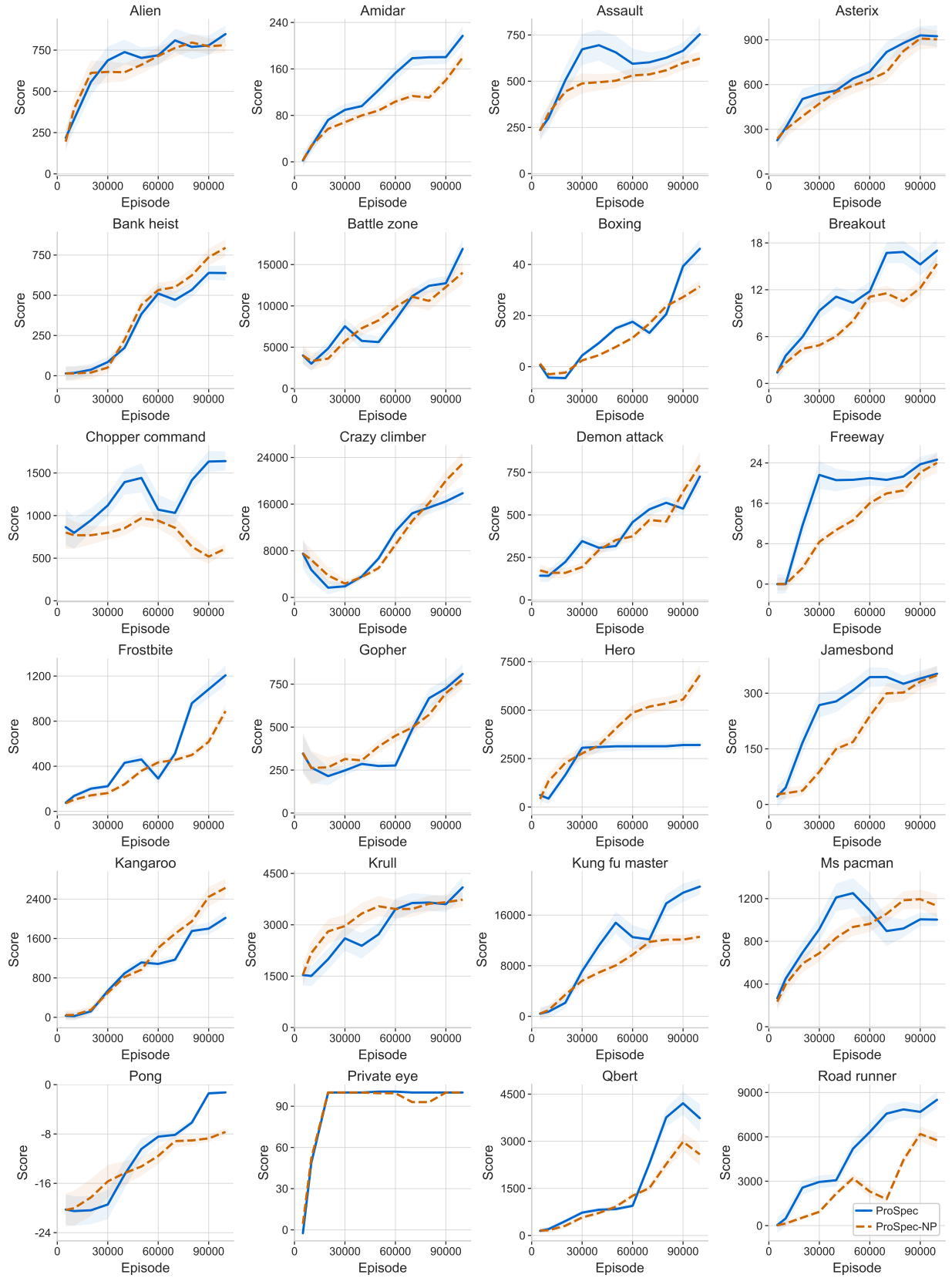


Fig. 5. The impact of ProSpec on data efficiency (Atari: Alien to Road Runner).

is approximately $\mathcal{O}(KH(C_{\text{FDM}} + C_Q))$, where C_{FDM} denotes the cost of one FDM transition prediction and C_Q

denotes the cost of one critic evaluation. In practice, the K candidate rollouts can be generated in parallel on GPUs.

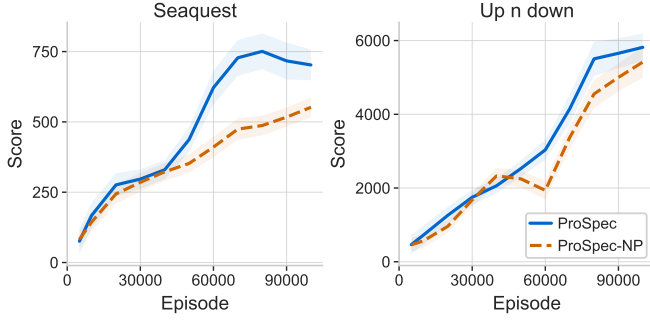


Fig. 6. The impact of ProSpec on data efficiency (Atari: Seaquest to up n down).

Therefore, the actual wall-clock overhead is mainly affected by the rollout horizon H , the number of candidates K , the batch size, and the efficiency of GPU parallelization.

To analyze the practical performance–efficiency trade-off, we profile ProSpec and PlayVirtual on DMControl under the same NVIDIA RTX 3090 GPU setting with a fixed random seed. Fig. 7 reports the evaluation reward curves and the wall-clock training time required to reach 100k environment steps. Overall, ProSpec introduces a moderate training-time overhead of approximately 10% to 20%, while achieving clear performance gains in several tasks. This result indicates that converting trial-and-error data collection into prospective value-aware interaction can improve data efficiency at the cost of additional training-time computation.

a) Task-dependent gains and overhead. Across the six DMControl tasks, the benefit of ProSpec is task-dependent. For long-horizon locomotion tasks such as *Walker*, *walk*, ProSpec achieves a substantial reward improvement of +432.4 points (+79%) with a 14% increase in wall-clock training time, from 17.9h to 20.3h. This result suggests that VPAF is particularly useful when early actions have delayed effects on future motion stability and control quality.

For medium-horizon control tasks such as *Cartpole*, *swingup*, ProSpec achieves a +29.5-point improvement with a 15% time overhead, showing that prospective value-aware interaction can improve multi-step control while keeping the additional cost moderate. For sparse-reward goal-reaching tasks such as *Reacher*, *easy*, ProSpec improves the reward by +188.6 points with a 20% increase in training time, indicating that evaluating candidate actions before real-environment execution is useful when immediate rewards provide limited guidance.

For dense-reward or relatively reactive tasks, such as *Ball in Cup*, *catch* and *Cheetah*, *run*, the baseline already learns effective behaviours quickly. In these cases, ProSpec introduces limited additional computation and achieves comparable peak performance, suggesting that the benefit of VPAF becomes more modest when trial-and-error interactions already provide informative feedback. For *Finger*, *spin*, ProSpec requires a longer training time but achieves similar final performance, indicating that the effectiveness of VPAF also depends on the quality and usefulness of the learned latent rollouts for each task.

b) Practical trade-off. These results show that ProSpec trades additional training-time computation for improved transition quality and data efficiency. This trade-off is favourable in long-horizon, sparse-reward, or difficult exploration tasks, where reducing low-value action execution can lead to clear performance gains. Importantly, the FDM-based prospective branch and VPAF are used only during training. During inference, ProSpec disables latent rollout generation and cumulative Q-value ranking, and actions are directly produced by the learned model-free policy or value network. Therefore, ProSpec introduces no additional inference-time planning overhead compared with the underlying model-free backbone.

c) Discussion on Atari. We do not directly profile the wall-clock overhead on Atari in this study. Nevertheless, the main additional operations of ProSpec remain FDM rollout generation and cumulative Q-value evaluation. Therefore, the training-time overhead is mainly controlled by the same factors, including K , H , model size, and GPU parallelization efficiency. Since VPAF is also disabled during Atari inference, ProSpec preserves model-free inference without additional online rollout computation. A more systematic profiling study across Atari games will be considered in future work.

D.5 Potential Societal Impact

Deep reinforcement learning (RL) has shown broad potential in domains such as gaming, robotics, healthcare, and conversational systems. However, many RL applications still suffer from low data efficiency and unstable training, especially when agents must learn from limited interactions or face delayed negative consequences of early actions. ProSpec addresses this issue by converting trial-and-error data collection into prospective value-aware interaction during training. Specifically, it uses FDM-generated latent prospective rollouts and VPAF to evaluate candidate actions before real-environment execution, thereby reducing low-value action execution and improving the quality of collected transitions while preserving model-free inference.

The potential societal benefit of ProSpec lies in improving the data efficiency and stability of RL training. By reducing unnecessary low-value interactions, ProSpec may lower the cost of training RL agents and improve learning efficiency in long-horizon, sparse-reward, or interaction-constrained tasks. This property is potentially useful for applications where environment interaction is costly or risky, such as robotics simulation, embodied decision-making, and interactive intelligent systems.

At the same time, ProSpec should be applied responsibly. Although the proposed method supports low-value-action-aware prospective filtering by reducing low-value action execution, it is not a substitute for constraint-based safe RL or formal safety verification. In high-stakes domains such as healthcare, autonomous systems, or human-facing decision support, additional safety constraints, human oversight, robustness evaluation, and domain-specific validation remain necessary. Moreover, when RL agents are trained from image-based observations, responsible AI policies should guide data collection, evaluation, and deployment to mitigate potential biases, distribution shifts, and unintended behaviors.

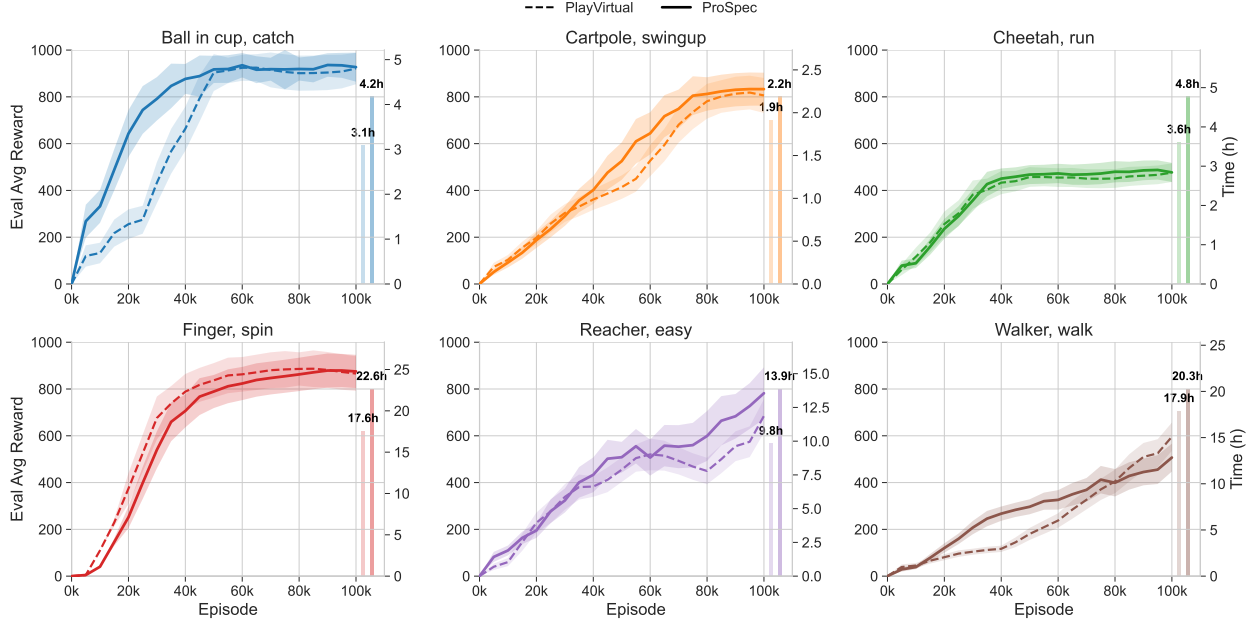


Fig. 7. ProSpec achieves higher rewards than PlayVirtual on most DMControl tasks with moderate additional training time. Curves show evaluation-average reward over environment steps, and bars indicate cumulative training time to 100K steps. All experiments are conducted on NVIDIA RTX 3090 GPUs with the same random seed.

Overall, ProSpec provides a training-time prospective value-aware interaction mechanism that may encourage further research on improving the quality of RL data collection under limited interaction budgets.

D.6 The Use of Large Language Models (LLMs)

Large Language Models (LLMs) were only used to assist in language polishing, such as grammar refinement and correction of typographical errors. They were not employed for idea generation, data analysis, or interpretation of results. All substantive contributions, including the conception, design, experiments, and writing of the manuscript, were carried out by the authors.

D.7 Reproducibility Statement

We have made every effort to ensure the reproducibility of our results. Implementation details of the model, training procedures, and hyperparameter settings are provided in Appendix. All datasets used in our experiments are publicly available, with preprocessing steps consistent with PlayVirtual [3]. To further facilitate reproduction, we provide our code and scripts as supplementary material at <https://github.com/liull2023/ProSpec>.

TABLE 10
Hyperparameters for DMControl.

Hyperparameter	Setting
Frame stack	3
Observation downsampling	(84, 84)
Augmentation	Random crop & intensity
Replay buffer size	100K
Initial exploration steps	1K
Action repeat	2 finger, spin and walker, walk; 8 cartpole, swingup; 4 otherwise
Training steps	100K
Evaluation trajectories	10
SAC Batch size	512
Q-function EMA τ	0.01
Critic target update freq	2
Discount factor	0.99
Initial temperature	0.1
Target network update period	1
Target network EMA τ	0.05
Actor & Critic & Encoder opt	
Optimizer	Adam
(β_1, β_2)	(0.9, 0.999)
Learning rate	0.0001
Temperature (α) opt Optimizer	
Optimizer	Adam
(β_1, β_2)	(0.5, 0.999)
Learning rate	0.0001
cycle consistency batch size	128
k (number of view angles)	3
t (number of prediction horizon)	6
M (number of virtual trajectories)	10
λ_{pred}	1.0
λ_c	1.0
warmup	Gaussian ramp-up ($i_{end} = 50K$)

TABLE 11
Hyperparameters for Atari.

Hyperparameter	Setting
Gray-scaling	True
Frame stack	4
Observation downsampling	(84, 84)
Augmentation	Random crop & intensity
Action repeat	4
Training steps	100K
Max frames per episode	108K
Reply buffer size	100K
Minimum replay size for sampling	2000
Mini-batch size	32
Optimizer	Adam
Optimizer:learning rate	0.0001
Optimizer: β_1	0.9
Optimizer: β_2	0.999
Optimizer: ϵ	0.00015
Max gradient norm	10
Update Distributional	Q
Dueling	True
Support of Q-distribution	51 bins
Discount factor	0.99
Reward clipping Frame stack	[-1, 1]
Priority exponent	0.5
Priority correction	$0.4 \rightarrow 1$
Exploration	Noisy nets
Noisy nets parameter	0.5
Evaluation trajectories	100
Replay period every	1
step Updates per step	2
Multi-step return length	10
Q network: channels	32, 64, 64
Q network: filter size	$8 \times 8, 4 \times 4, 3 \times 3$
Q network: stride	4, 2, 1
Q network: hidden units	256
Target network update period	1
τ (EMA coefficient)	0
Additional Hyperparameters in ProSpec	
M (number of virtual trajectories)	$2 \mathcal{A} $ (two times of action space size)
λ_{pred} (weight for prediction loss)	1
λ_c^{max} (a weight related to cyclical consistency loss)	1
k (number of view angles)	9
t (number of prediction horizon)	3
λ_{pred}	1.0
λ_c	1.0
warmup	Gaussian ramp-up ($i_{end} = 50K$)

REFERENCES

- [1] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel *et al.*, “Soft actor-critic algorithms and applications,” *arXiv preprint arXiv:1812.05905*, 2018.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, “Human-level control through deep reinforcement learning,” *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [3] T. Yu, C. Lan, W. Zeng, M. Feng, Z. Zhang, and Z. Chen, “Playvirtual: Augmenting cycle-consistent virtual trajectories for reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 5276–5289, 2021.
- [4] Y. Yue, B. Kang, Z. Xu, G. Huang, and S. Yan, “Value-consistent representation learning for data-efficient reinforcement learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 9, 2023, pp. 11 069–11 077.
- [5] M. Laskin, A. Srinivas, and P. Abbeel, “Curl: Contrastive unsupervised representations for reinforcement learning,” in *International conference on machine learning*. PMLR, 2020, pp. 5639–5650.
- [6] M. Schwarzer, A. Anand, R. Goel, R. D. Hjelm, A. Courville, and P. Bachman, “Data-efficient reinforcement learning with self-predictive representations,” in *International Conference on Learning Representations*, 2021.
- [7] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. Richemond, E. Buchatskaya, C. Doersch, B. Avila Pires, Z. Guo, M. Gheshlaghi Azar *et al.*, “Bootstrap your own latent—a new approach to self-supervised learning,” *Advances in neural information processing systems*, vol. 33, pp. 21 271–21 284, 2020.
- [8] A. Zhang, R. T. McAllister, R. Calandra, Y. Gal, and S. Levine, “Learning invariant representations for reinforcement learning without reconstruction,” 2021.
- [9] L. Dinh, J. Sohl-Dickstein, and S. Bengio, “Density estimation using real nvp,” in *International Conference on Learning Representations*, 2022.
- [10] L. Huang, X. Liu, B. Lang, A. Yu, Y. Wang, and B. Li, “Orthogonal weight normalization: Solution to optimization over multiple dependent stiefel manifolds in deep neural networks,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.